



UNIVERSIDAD DE ESPECIALIDADES ESPÍRITU SANTO
MAESTRÍA EN INTELIGENCIA ARTIFICIAL

ASIGNATURA:

PROYECTO INTEGRADOR EN INTELIGENCIA ARTIFICIAL

ACTIVIDAD:

RECURSOS Y ESTADO DEL ARTE

Sistema de BI conversacional con IA: Text-to-SQL, robustez y optimización

AUTORES:

CABRERA PARRALES JOEL DAVID
SÁNCHEZ ARÉVALO ANÍBAL ANDRÉS

TUTOR:

MG. ALEXANDRA J. ARCINIEGAS C.

SAMBORONDÓN – ECUADOR

ABRIL – 2026

1. Mapa del Estado del Arte

1.1. Estrategia de búsqueda bibliográfica

La revisión de la literatura se condujo siguiendo una estrategia sistemática inspirada en las directrices de Kitchenham y Charters (2007) para revisiones técnicas en ciencias de la computación. El proceso se organizó en cuatro fases consecutivas: formulación de la ecuación de búsqueda, selección de bases de datos, aplicación de criterios de inclusión y exclusión, y depuración mediante lectura en tres capas (título, resumen, texto completo).

1.1.1. Ecuación de búsqueda

La ecuación principal combina los tres dominios centrales del proyecto mediante operadores booleanos:

(“text-to-SQL” OR “natural language to SQL” OR “semantic parsing” OR “conversational BI”) AND (“difficulty classification” OR “robustness” OR “adversarial” OR “hyperparameter optimization” OR “LLM” OR “large language model”)

Se aplicó una ecuación complementaria para el componente de fairness y sesgo algorítmico:

(“algorithmic fairness” OR “bias detection” OR “disparate impact”) AND (“machine learning” OR “NLP” OR “classification”)

1.1.2. Bases de datos consultadas

- **ACM Digital Library:** acceso directo a las actas de SIGMOD, KDD y CSCW, así como a ACM Computing Surveys.
- **IEEE Xplore:** conferencias IEEE sobre Knowledge and Data Engineering (TKDE), ICCV y bases de datos.
- **Scopus:** índice multidisciplinario con métricas de citación para evaluar impacto.
- **arXiv:** preprints de NLP, ML y IA; fuente primaria para trabajos recientes aún no publicados en revista.
- **Google Scholar:** búsqueda complementaria para detectar trabajos no indexados en las bases anteriores.
- **Actas de conferencias especializadas:** EMNLP, ACL, AAAI, NeurIPS, NAACL e ICLR (2018–2025).

1.1.3. Criterios de inclusión y exclusión

Criterios de inclusión	Criterios de exclusión
Publicaciones en inglés o español (2018–2025)	Trabajos duplicados entre bases de datos
Artículos con revisión por pares o preprints con >50 citas	Resúmenes sin texto completo accesible
Trabajos sobre datasets públicos reproducibles	Publicaciones sin métricas cuantitativas
Enfoque metodológico claro y resultados verificables	Tesis de grado no arbitradas ni publicadas
Pertinencia directa a Text-to-SQL, robustez NLP o fairness	Trabajos sobre idiomas distintos al inglés sin evaluación cross-lingual

Tabla 1. Criterios de inclusión y exclusión aplicados en la revisión bibliográfica. Elaboración propia basada en Kitchenham y Charters (2007).

De los aproximadamente 200 trabajos identificados en la búsqueda inicial se aplicó una depuración progresiva: 110 pasaron el filtro por título, 48 pasaron el filtro por resumen y 24 fueron seleccionados tras lectura completa. Estas 24 referencias constituyen el corpus central del estado del arte presentado a continuación.

1.2. Ejes temáticos del estado del arte

La literatura revisada se organiza en seis ejes temáticos que responden a las dimensiones del proyecto. La Tabla 2 presenta una síntesis estructurada de los trabajos más representativos con sus contribuciones y limitaciones identificadas.

Eje temático	Trabajos clave	Contribución principal	Limitación identificada
A. Datasets y benchmarks Text-to-SQL	<i>Yu et al. (2018); Zhong et al. (2017); Lee et al. (2021)</i>	Spider: 10.181 preguntas cross-domain, 200 BDs. WikiSQL: gran volumen single-table. KaggleDBQA: datos reales.	Spider tiene solo 10 ejemplos “extra” en validación; desbalance extremo que afecta métricas macro.
B. Arquitecturas de parsing semántico	<i>Wang et al. (2020); Scholak et al. (2021); Lin et al. (2020); Li et al. (2024)</i>	RAT-SQL, PICARD y BRIDGE introducen atención sobre el esquema. DIN-SQL usa LLMs con descomposición de tareas.	No clasifican la dificultad previamente; asumen costo computacional uniforme por consulta.
C. LLMs aplicados a Text-to-SQL	<i>Rajkumar et al. (2022); Pourreza & Rafiei (2024); Gao et al. (2024)</i>	GPT-4, DIN-SQL y DAIL-SQL demuestran que LLMs con prompt engineering alcanzan resultados competitivos sin fine-tuning.	Dependencia de APIs comerciales costosas; latencia variable; sin evaluación de robustez adversarial.
D. Robustez y ataques	<i>Morris et al. (2020); Jin et al. (2020); Zang et al.</i>	TextAttack, TextFooler establecen la métrica ASR.	Dr.Spider se centra en parsers, no en clasificadores de dificultad.

Eje temático	Trabajos clave	Contribución principal	Limitación identificada
adversariales en NLP	<i>(2020); Deng et al. (2022)</i>	Dr.Spider evalúa robustez específica de parsers T2SQL.	Falta evaluación de estrés compuesto.
E. Data leakage y diagnóstico de pipelines	<i>Kaufman et al. (2012); Zech et al. (2018); Gencoglu et al. (2019)</i>	Criterios sistemáticos para detectar fuga de información. Zech demuestra leakage en modelos médicos con imágenes.	Raramente se aplican como filtro obligatorio en trabajos recientes de Text-to-SQL.
F. Optimización e interpretabilidad de HP	<i>Friedman (2001); Probst et al. (2019); Moosbauer et al. (2021)</i>	PDP, ranking de importancia y análisis de interacciones entre hiperparámetros de Random Forest.	Aplicados a tareas tabulares clásicas; escasa aplicación a pipelines NLP con representaciones TF-IDF.

Tabla 2. Mapa estructurado del estado del arte. Seis ejes temáticos con trabajos representativos, contribución y limitación. Elaboración propia.

1.3. Análisis de tendencias

La revisión identifica cuatro tendencias que condicionan el diseño del proyecto:

- **Migración hacia LLMs como generadores SQL:** desde 2022 la comunidad ha transitado de modelos especializados (RAT-SQL, PICARD) hacia el uso de LLMs de propósito general (GPT-4, Qwen, Llama) con prompt engineering sofisticado. Rajkumar et al. (2022) demostraron que Codex alcanza 67% de exactitud en Spider sin fine-tuning, y Pourreza y Rafiei (2024) con DIN-SQL superan el 85% descomponiendo la tarea en sub-problemas. Sin embargo, estas aproximaciones dependen de APIs comerciales con costos variables y sin garantías de privacidad, lo que motiva nuestra elección de modelos locales con Ollama.
- **Creciente preocupación por la robustez:** el trabajo de Deng et al. (2022) con Dr.Spider marca un punto de inflexión al demostrar que los parsers Text-to-SQL que lideran benchmarks limpios sufren caídas significativas ante perturbaciones realistas (sinónimos, paráfrasis, cambios de esquema). No obstante, la evaluación de robustez se centra en la exactitud del SQL generado, no en tareas auxiliares como la clasificación de dificultad, lo que constituye un espacio no explorado.
- **Fairness algorítmico como requisito emergente:** Mehrabi et al. (2021) documentan que los sistemas de ML aplicados a dominios sensibles requieren auditoría de equidad. Barocas et al. (2019) formalizan los criterios de Disparate Impact para sistemas automatizados. La aplicación de estos marcos a clasificadores Text-to-SQL es prácticamente inexistente, lo que representa una oportunidad de contribución.
- **Interpretabilidad del tuning como práctica incipiente:** Moosbauer et al. (2021) proponen el uso de PDP para explicar decisiones de optimización de hiperparámetros, pero

su aplicación se limita a tareas tabulares. La transferencia de estas técnicas a pipelines NLP con representaciones dispersas (TF-IDF) no ha sido documentada.

1.4. Identificación y justificación de la brecha

La lectura cruzada de los seis ejes revela una brecha concreta y bien delimitada que fundamenta la pertinencia del presente proyecto. La brecha se articula en tres dimensiones complementarias:

- **Dimensión 1: Clasificación de dificultad sin data leakage:** no existe en la literatura un trabajo que construya un clasificador de dificultad SQL usando exclusivamente la pregunta en lenguaje natural como entrada, con verificación explícita de ausencia de fuga de información. Yu et al. (2018) definen las etiquetas de dificultad, pero no las explotan como variable objetivo predictiva. Los trabajos posteriores que usan Spider se centran en la generación SQL, no en la clasificación previa de complejidad.
- **Dimensión 2: Evaluación integral de robustez y fairness:** los frameworks de robustez existentes (TextAttack, Dr.Spider) evalúan parsers completos, no clasificadores auxiliares. La combinación de robustez adversarial (ASR), análisis de drift, pruebas de estrés compuesto y auditoría de fairness (Disparity Ratio) sobre el mismo pipeline experimental no ha sido reportada.
- **Dimensión 3: Interpretabilidad del tuning en NLP:** la aplicación de PDP, ranking de importancia con métodos múltiples y análisis de interacciones entre hiperparámetros a un Random Forest sobre representaciones TF-IDF en el dominio Text-to-SQL es, hasta donde alcanza esta revisión, una combinación no explorada.

Estas tres dimensiones justifican la pertinencia del proyecto: el trabajo propuesto llena el vacío construyendo un pipeline completo que integra clasificación sin leakage, evaluación de robustez y fairness, y optimización interpretable de HP, todo sobre el benchmark Spider y en el contexto de un sistema de BI conversacional funcional.

1.5. Redacción comparativa de los trabajos centrales

Los sistemas de generación SQL han evolucionado en tres generaciones. La primera, representada por Zhong et al. (2017) con Seq2SQL, abordó el problema como traducción secuencia a secuencia sobre WikiSQL, un dataset de tabla única que simplifica artificialmente la tarea. La segunda generación, con Wang et al. (2020) y Scholak et al. (2021), incorporó mecanismos de atención sobre el esquema relacional (RAT-SQL) y decodificación restringida por gramática (PICARD), logrando avances significativos sobre Spider. La tercera generación, emergente desde 2022,

aprovecha LLMs preentrenados con estrategias de prompt engineering: Rajkumar et al. (2022) demostraron la viabilidad de Codex sin fine-tuning, Pourreza y Rafiei (2024) con DIN-SQL introdujeron la descomposición de la tarea en sub-problemas clasificados por dificultad (aunque usando el propio LLM como clasificador, no un modelo independiente), y Gao et al. (2024) con DAIL-SQL optimizaron la selección de ejemplos few-shot.

Frente a esta evolución, la evaluación de robustez ha ido rezagada. Morris et al. (2020) establecieron el framework TextAttack como estándar para ataques word-level, Jin et al. (2020) lo aplicaron a BERT en tareas de clasificación binaria (SST-2, AG News) y Zang et al. (2020) propusieron ataques por optimización combinatoria. El trabajo más cercano a nuestro dominio es Dr.Spider (Deng et al., 2022), que evalúa la robustez de parsers Text-to-SQL ante 17 tipos de perturbaciones, pero se centra en la exactitud del SQL generado y no aborda clasificadores auxiliares ni métricas de equidad.

En la dimensión de interpretabilidad, Friedman (2001) establece el concepto de PDP como herramienta para visualizar efectos marginales de variables. Probst et al. (2019) aplican PDP al análisis de hiperparámetros de Random Forest, identificando que `max_depth` y `mtry` son consistentemente los más influyentes. Moosbauer et al. (2021) articulan el enfoque dentro del campo de explainable AutoML en NeurIPS. La aplicación conjunta de estos tres referentes sobre un pipeline TF-IDF + RF aplicado a la clasificación de dificultad SQL constituye la contribución metodológica diferenciadora del presente proyecto.

2. Alcance del Proyecto

2.1. Cronograma basado en metodología ágil

El proyecto se organiza en cuatro sprints de cuatro semanas cada uno, totalizando dieciséis semanas de ejecución. La elección de Scrum adaptado (Schwaber y Sutherland, 2020) responde a la naturaleza iterativa del trabajo: los hallazgos de cada sprint (como la detección de data leakage en el Sprint 2) condicionan las decisiones de los sprints siguientes. La Tabla 3 detalla la planificación.

Sprint	Semanas	Tema central	Actividades y entregable
1	1 a 2	Preparación	Revisión bibliográfica, estado del arte, descarga y exploración de Spider, diseño de la arquitectura de la app BI, setup del repositorio. Entregable: documento de estado del arte + baseline reproducible.
2	3 a 4	Evaluación de modelos	Benchmarking 10 modelos, auditoría de fairness (DI Ratio, EO Gap), diagnóstico overfitting/underfitting, detección del data leakage, redefinición del pipeline. Desarrollo del backend Express + Prisma + Ollama. Entregable: informe de evaluación + API funcional.
3	4 a 6	Simulación y robustez	9 escenarios de perturbación (ruido, drift, adversarial, compuesto), matriz FMEA. Desarrollo del frontend Angular con chat conversacional y 22 tipos de gráficos. Entregable: workshop de robustez + app web funcional.
4	6 a 8	Optimización y cierre	Grilla 120 experimentos HP, PDP, ranking, interacciones. Integración del clasificador al pipeline de producción. Informe consolidado y defensa. Entregable: informe final + presentación + demo en vivo.

Tabla 3. Cronograma del proyecto en cuatro sprints ágiles. Cada sprint genera entregables verificables. Elaboración propia.

Las ceremonias Scrum se adaptan al contexto académico: planning al inicio de cada sprint, standups asincrónicos tres veces por semana, sprint review con la tutora al cierre, y retrospectiva interna del equipo.

2.2. Plan de recursos

2.2.1. Recursos humanos

Persona	Rol preferente	Responsabilidades
Cabrera Parrales Joel David	<i>Desarrollo técnico</i>	Implementación backend (Express, Prisma, Ollama), pipeline de IA, ejecución de experimentos, generación de figuras.
Sánchez Arévalo Aníbal Andrés	<i>Análisis y redacción</i>	Análisis crítico de resultados, desarrollo frontend (Angular, ApexCharts), redacción académica, gestión bibliográfica.
Mg. Alexandra J. Arciniegas C.	<i>Dirección académica</i>	Validación metodológica, revisión de avances, retroalimentación por sprint, aprobación de entregables.

Tabla 4. Asignación de recursos humanos. Dedicación estimada: 15 horas semanales por autor durante 16 semanas. Elaboración propia.

2.2.2. Recursos tecnológicos

- **Backend:** TypeScript + Node.js + Express 5, Prisma ORM 7.6, PostgreSQL 16. Arquitectura Clean Architecture con separación de capas Domain/Application/Infrastructure/Presentation.
- **Frontend:** Angular 17+ con componentes standalone, Tailwind CSS, ng-apexcharts (22 tipos de visualización).
- **Motor de IA:** Ollama (ejecución local de LLMs). Modelo SQL: Qwen 3.5 397B (cloud) / SQLCoder 7B (local). Modelo de análisis: Qwen 2.5 Coder 7B.
- **Investigación:** Python 3.10, scikit-learn 1.3, pandas, numpy, matplotlib, seaborn, Jupyter Notebook.
- **Control de versiones:** Git + GitHub con estructura profesional de repositorio.
- **Hardware:** Laptops personales (16 GB RAM). Sin requerimiento de GPU: el pipeline de investigación corre en CPU en ~8 minutos para 120 experimentos.

2.2.3. Recursos de datos

- **Dataset principal:** Spider Benchmark (Yu et al., 2018): 10.181 ejemplos, 200 BDs, 138 dominios. Descarga pública desde HuggingFace Hub, ~50 MB comprimido.
- **Dataset de demostración:** AdventureWorks para PostgreSQL: base de datos empresarial con esquemas de ventas, producción, RRHH. Permite demostrar la app BI sobre datos realistas.

2.3. Definición de hitos y entregables

ID	Semana	Entregable	Criterio de aceptación
H1	Sem 2	Estado del arte + baseline inicial + repositorio GitHub	Documento con ≥ 20 refs APA, modelo con $F1 > 0,30$, código reproducible.
H2	Sem 3	Informe de evaluación de modelos + API backend funcional	10 modelos evaluados, fairness auditado, leakage documentado, endpoints de auth y chatbot operativos.
H3	Sem 5	Workshop de robustez + app web completa	9 escenarios simulados, RSI calculado, chat conversacional con generación de gráficos en tiempo real.
H4	Sem 7	Informe consolidado + presentación + demo en vivo	Documento con 2 capítulos integrados, PPTX de defensa, sistema demostrable ante tribunal.

Tabla 5. Hitos del proyecto con entregables tangibles y criterios de aceptación verificables. Elaboración propia.

3. Referencias Bibliográficas

- Barocas, S., Hardt, M., & Narayanan, A. (2019). Fairness and machine learning: Limitations and opportunities. fairmlbook.org. <https://fairmlbook.org/>
- Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>
- Deng, X., Awadallah, A. H., Meek, C., Polozov, O., Sun, H., & Richardson, M. (2022). Structure-grounded pretraining for text-to-SQL. En Proceedings of the 2022 Conference of the NAACL (pp. 1786–1798). Association for Computational Linguistics.
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. The Annals of Statistics, 29(5), 1189–1232. <https://doi.org/10.1214/aos/1013203451>
- Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A survey on concept drift adaptation. ACM Computing Surveys, 46(4), 1–37. <https://doi.org/10.1145/2523813>
- Gao, D., Wang, H., Li, Y., Sun, X., Qian, Y., Ding, B., & Zhou, J. (2024). Text-to-SQL empowered by large language models: A benchmark evaluation. Proceedings of the VLDB Endowment, 17(5), 1132–1145.
- Gencoglu, O., van Gils, M., Guldogan, E., Morikawa, C., Süzen, M., Gruber, M., Leinonen, J., & Huttunen, H. (2019). HARK side of deep learning. arXiv preprint arXiv:1904.07633.
- Jin, D., Jin, Z., Zhou, J. T., & Szolovits, P. (2020). Is BERT really robust? Proceedings of the AAAI Conference on Artificial Intelligence, 34(5), 8018–8025. <https://doi.org/10.1609/aaai.v34i05.6311>
- Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining. ACM Transactions on Knowledge Discovery from Data, 6(4), 1–21. <https://doi.org/10.1145/2382577.2382579>
- Kitchenham, B., & Charters, S. (2007). Guidelines for performing systematic literature reviews in software engineering. Keele University Technical Report EBSE-2007-01.
- Lee, C. H., Polozov, O., & Richardson, M. (2021). KagggleDBQA: Realistic evaluation of text-to-SQL parsers. En Proceedings of the 59th Annual Meeting of the ACL (pp. 2261–2273). <https://doi.org/10.18653/v1/2021.acl-long.176>

- Li, J., Hui, B., Qu, G., Yang, J., Li, B., Li, B., Wang, B., Qin, B., Geng, R., Huo, N., Zhou, X., Ma, C., Li, G., Chang, K., Huang, F., Cheng, R., & Li, Y. (2024). Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQLs. En *Advances in Neural Information Processing Systems (NeurIPS)*, 36.
- Lin, X. V., Socher, R., & Xiong, C. (2020). Bridging textual and tabular data for cross-domain text-to-SQL. En *Findings of EMNLP 2020* (pp. 4870–4888). <https://doi.org/10.18653/v1/2020.findings-emnlp.438>
- Mehrabi, N., Morstatter, F., Saxena, N., Lerman, K., & Galstyan, A. (2021). A survey on bias and fairness in machine learning. *ACM Computing Surveys*, 54(6), 1–35. <https://doi.org/10.1145/3457607>
- Moosbauer, J., Herbringer, J., Casalicchio, G., Lindauer, M., & Bischl, B. (2021). Explaining hyperparameter optimization via partial dependence plots. En *Advances in Neural Information Processing Systems (NeurIPS)*, 34, 2280–2291.
- Morris, J. X., Lifland, E., Yoo, J. Y., Grigsby, J., Jin, D., & Qi, Y. (2020). TextAttack: A framework for adversarial attacks in NLP. En *Proceedings of the 2020 EMNLP: System Demonstrations* (pp. 119–126). <https://doi.org/10.18653/v1/2020.emnlp-demos.16>
- Pourreza, M., & Rafiei, D. (2024). DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. En *Advances in Neural Information Processing Systems (NeurIPS)*, 36.
- Probst, P., Wright, M. N., & Boulesteix, A. L. (2019). Hyperparameters and tuning strategies for random forest. *WIREs Data Mining and Knowledge Discovery*, 9(3), e1301. <https://doi.org/10.1002/widm.1301>
- Rajkumar, N., Li, R., & Baber, D. (2022). Evaluating the text-to-SQL capabilities of large language models. *arXiv preprint arXiv:2204.00498*.
- Scholak, T., Schucher, N., & Bahdanau, D. (2021). PICARD: Parsing incrementally for constrained auto-regressive decoding. En *Proceedings of the 2021 EMNLP* (pp. 9895–9901). <https://doi.org/10.18653/v1/2021.emnlp-main.779>
- Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide: The definitive guide to Scrum, the rules of the game*. Scrum.org.
- Wang, B., Shin, R., Liu, X., Polozov, O., & Richardson, M. (2020). RAT-SQL: Relation-aware schema encoding. En *Proceedings of the 58th ACL* (pp. 7567–7578). <https://doi.org/10.18653/v1/2020.acl-main.677>
- Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., & Radev, D. (2018). Spider: A large-scale human-labeled dataset for text-to-SQL. En *Proceedings of the 2018 EMNLP* (pp. 3911–3921). <https://doi.org/10.18653/v1/D18-1425>
- Zang, Y., Qi, F., Yang, C., Liu, Z., Zhang, M., Liu, Q., & Sun, M. (2020). Word-level textual adversarial attacking as combinatorial optimization. En *Proceedings of the 58th ACL* (pp. 6066–6080). <https://doi.org/10.18653/v1/2020.acl-main.540>
- Zech, J. R., Badgeley, M. A., Liu, M., Costa, A. B., Titano, J. J., & Oermann, E. K. (2018). Variable generalization performance of a deep learning model. *PLoS Medicine*, 15(11), e1002683. <https://doi.org/10.1371/journal.pmed.1002683>
- Zhong, V., Xiong, C., & Socher, R. (2017). Seq2SQL: Generating structured queries from natural language using reinforcement learning. *arXiv preprint arXiv:1709.00103*.